

Chapter one

1. Random Variables

A random variable, usually written X , is a variable whose possible values are numerical outcomes of a random phenomenon. There are two types of random variables, discrete and continuous. All random variables have a cumulative distribution function. It is a function that gives the probability that the random variable X is less than or equal to x for every value of x .

1.1. Discrete Random Variables

A **discrete random** variable is one that may take on only a countable number of distinct values, such as 0,1,2,3,4,..... If a random variable can take only a **finite** number of distinct values, then it must be discrete. Examples of discrete random variables include the number of children in a family, the number of defective light bulbs in a box of ten. The **probability distribution** of a discrete random variable is a list of probabilities associated with each of its possible values. It is also sometimes called the probability function or the probability mass function.

When the sample space Ω has a finite number of equally likely outcomes, so that the discrete uniform probability law applies. Then, the probability of any event x is given by:

$$P(A) = \frac{\text{number of elements of } x}{\text{number of elements of } \Omega}$$

This distribution may also be described by the probability histogram. Suppose a random variable X may take k different values, with the probability that $X = x_i$ defined to be $P(X = x_i) = P_i$. The probabilities P_i must satisfy the following:

1. $0 < P_i < 1$ for each i .
2. $P_1 + P_2 + P_3 + \cdots + P_k = 1$ or $\sum_{i=1}^k p_i = 1$

Example 1: Suppose a variable X can take the values 1, 2, 3, or 4. The probabilities associated with each outcome are described by the following table:

Outcome	1	2	3	4
Probability	0.1	0.3	0.4	0.2

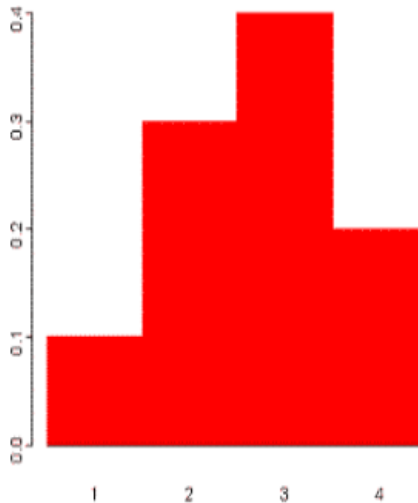


Figure 1: Probability distribution histogram.

The cumulative distribution function for the above probability distribution is calculated as follows:

The probability that X is less than or equal to 1 is 0.1,

The probability that X is less than or equal to 2 is $0.1+0.3 = 0.4$,

The probability that X is less than or equal to 3 is $0.1+0.3+0.4 = 0.8$,

and the probability that X is less than or equal to 4 is $0.1+0.3+0.4+0.2 = 1$.

1.2.Continuous Random Variables

A continuous random variable is one which takes an infinite number of possible values. Continuous random variables are usually measurements. Examples include height, weight and the amount of sugar in an orange. A continuous random variable is not defined at specific values. Instead, it is defined over an interval of values, and is represented by the area under a curve. The curve, which represents a function $p(x)$, must satisfy the following:

1. The curve has no negative values ($p(x) > 0$ for all x).
2. The total area under the curve is equal to 1.

A curve meeting these requirements is known as a density curve. If any interval of numbers of equal width has an equal probability, then the curve describing the distribution is a rectangle, with constant height across the interval and 0 height elsewhere, these curves are known as uniform distributions.

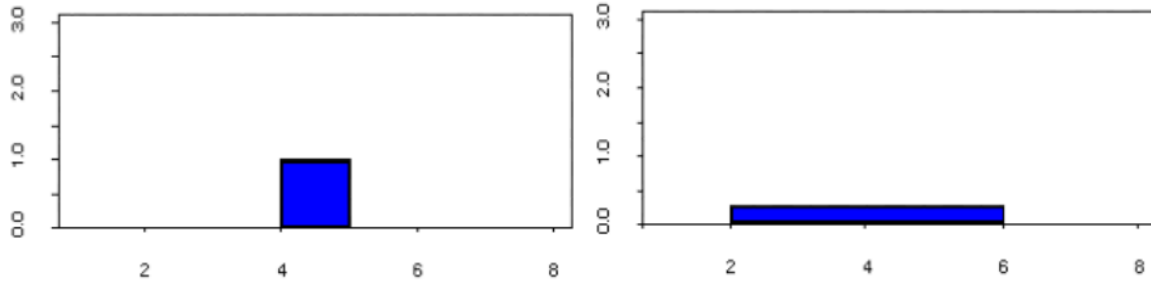


Figure 2: Uniform distribution.

Another type of distribution is the normal distribution, having a bell-shaped density curve described by its mean μ and standard deviation σ . The height of a normal density curve at a given point x is given by:

$$h = \frac{1}{\sigma\sqrt{2\pi}} e^{-0.5\left(\frac{x-\mu}{\sigma}\right)^2}$$

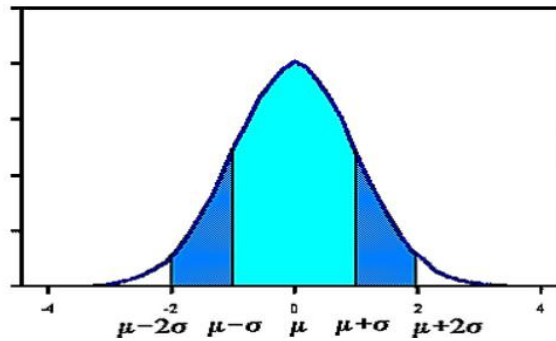


Figure 3: The Standard normal curve.

2. Joint Probability

Joint probability is the probability of event Y occurring at the same time event X occurs. Its notation is $P(X \cap Y)$ or $P(X, Y)$, which reads: the joint probability of X and Y .

$$P(X, Y) = P(x) \times P(Y)$$

If X and Y are discrete random variables, then $f(x, y)$ must satisfy: $0 \leq f(x, y) \leq 1$ and $\sum_x \sum_y f(x, y) = 1$.

If X and Y are continuous random variables, then $f(x, y)$ must satisfy: $f(x, y) \geq 0$ and, $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) = 1$.

Example 2: For discrete random variable, if the probability of rolling a four on one die is $P(X)$ and if the probability of rolling a four on second die is $P(Y)$. Find $P(X, Y)$.

Solution:

We have $P(X) = P(Y) = 1/6$

$$P(X, Y) = P(X) \times P(Y) = \frac{1}{6} \times \frac{1}{6} = \frac{1}{36} = 0.0277 = 2.8\%$$

3. Conditional Probabilities

It happens when there are dependent events. We have to use the symbol "|" to mean "given":

1. $P(B|A)$ means "Event B given Event A has occurred".
2. $P(B|A)$ is also called the "Conditional Probability" of B given A has occurred.
3. And we write it as

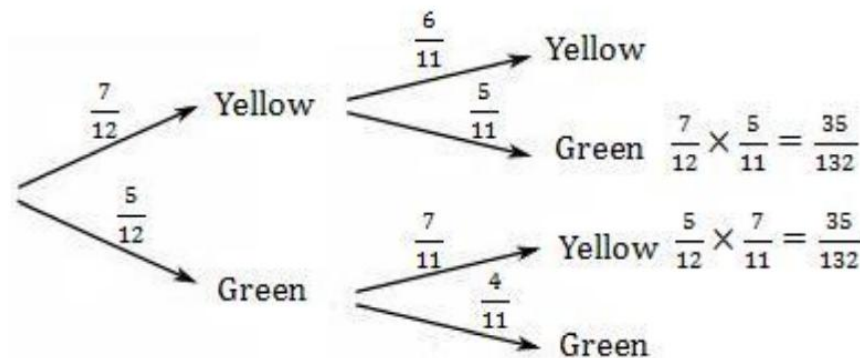
$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Where $P(B) > 0$.

Example 3: A box contains 5 green pencils and 7 yellow pencils. Two pencils are chosen at random from the box without replacement. What is the probability that they are different colors?

Solution:

Using a tree diagram

**4. Independence of Two Variables**

The concept of independent random variables is very similar to independent events. If two events A and B are independent, we have $P(A, B) = P(A)P(B) = P(A \cap B)$. For example, let's say you wanted to know the average weight of a bag of sugar so you randomly sample 50 bags from various grocery

stores. You wouldn't expect the weight of one bag to affect another, so the variables are independent.

5. Model of information transmission system

Transmitting a message from a transmitter to a receiver can be sketched as in Fig. 7: The components of information system as described by Shannon are:

1. An information source is a device which randomly delivers symbols from an alphabet. As an example, a PC (Personal Computer) connected to internet is an information source which produces binary digits from the binary alphabet $\{0, 1\}$.
2. A source encoder allows one to represent the data source more compactly by eliminating redundancy: it aims to reduce the data rate.
3. A channel encoder adds redundancy to protect the transmitted signal against transmission errors.
4. A channel is a system which links a transmitter to a receiver. It includes signaling equipment and pair of copper wires or coaxial cable or optical fiber, among other possibilities.
5. The rest of blocks is the receiver end, each block has inverse processing to the corresponding transmitted end.

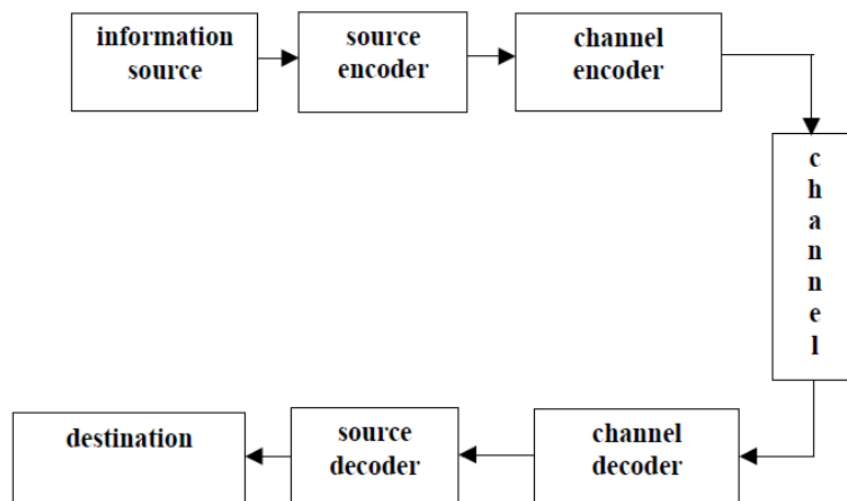


Figure 4: Shannon paradigm

6. Self- information

In information theory, self-information is a measure of the information content associated with the outcome of a random variable. It is expressed in a unit of information, for example bits, nats, or hartleys, depending on the base of the logarithm used in its calculation.

A bit is the basic unit of information in computing and digital communications. A bit can have only one of two values, and may therefore be physically implemented with a two-state device. These values are most commonly represented as 0 and 1.

The amount of self-information contained in a probabilistic event depends only on the probability of that event: the smaller its probability, the larger the self-information associated with receiving the information that the event indeed occurred as shown in Fig.8.

- i- Information is zero if $P(x_i) = 1$ (certain event)
- ii- Information increase as $P(x_i)$ decrease to zero
- iii- Information is a +ve quantity.

The log function satisfies all previous three points hence:

$$I(x_i) = \log_a \frac{1}{P(x_i)} = -\log_a(P(x_i))$$

Where $I(x_i)$ is self-information of (x_i) .

Example 4: A fair die is thrown, find the amount of information gained if you are told that 4 will appear.

Solution:

$$P(1) = P(2) = \dots \dots \dots = P(6) = \frac{1}{6}$$

$$I(4) = -\log_2 \left(\frac{1}{6} \right) = \frac{\ln(\frac{1}{6})}{\ln 2} = 2.5849 \text{ bits}$$

Example 5: A biased coin has $P(\text{Head})=0.3$. Find the amount of information gained if you are told that a tail will appear.

Solution:

$$P(\text{tail}) = 1 - P(\text{head}) = 1 - 0.3 = 0.7$$

$$I(\text{tail}) = -\log_2 0.7 = 0.5145 \text{ bits}$$

7. Average information (entropy)

In information theory, entropy is the average amount of information contained in each message received. Here, message stands for an event, sample or character drawn from a distribution or data stream. Entropy thus characterizes our uncertainty about our source of information.

7.1.Source Entropy

If the source produces not equiprobable messages then $I(x_i)$, $i = 1, 2, \dots, n$ are different. Then the statistical average of $I(x_i)$ over i will give the average amount of uncertainty associated with source X . This average is called source entropy and denoted by $H(X)$, given by:

$$H(X) = \sum_{i=1}^n P(x_i) I(x_i)$$

$$\therefore H(X) = - \sum_{i=1}^n P(x_i) \log_a P(x_i)$$

Example 6: Find the entropy of the source producing the following messages:

$$Px_1 = 0.25, \quad Px_2 = 0.1, \quad Px_3 = 0.15, \quad \text{and } Px_4 = 0.5$$

Solution:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_a P(x_i)$$

$$= - \frac{[0.25 \ln 0.25 + 0.1 \ln 0.1 + 0.15 \ln 0.15 + 0.5 \ln 0.5]}{\ln 2}$$

$$H(X) = 1.7427 \frac{\text{bits}}{\text{symbol}}$$

7.2.Source Entropy Rate

It is the average rate of amount of information produced per second.

$$R(X) = H(X) \times \text{rate of producing the symbols} = \frac{\text{bits}}{\text{sec}} = \text{bps}$$

The unit of $H(X)$ is bits/symbol and the rate of producing the symbols is symbol/sec, so that the unit of $R(X)$ is bits/sec. Sometimes:

$$R(X) = \frac{H(X)}{\bar{\tau}},$$

Where:

$$\bar{\tau} = \sum_{i=1}^n \tau_i P(x_i)$$

$\bar{\tau}$ is the average time duration of symbols, τ_i is the time duration of the symbol x_i .

Example 7: A source produces dots ‘.’ And dashes ‘-’ with $P(\text{dot})=0.65$. If the time duration of dot is 200ms and that for a dash is 800ms. Find the average source entropy rate.

Solution:

$$P(\text{dash}) = 1 - P(\text{dot}) = 1 - 0.65 = 0.35$$

$$H(X) = -[0.65 \log_2(0.65) + 0.35 \log_2(0.35)] = 0.934 \text{ bits/symbol}$$

$$\bar{\tau} = 0.2 \times 0.65 + 0.8 \times 0.35 = 0.41 \text{ sec}$$

$$R(X) = \frac{H(X)}{\bar{\tau}} = \frac{0.934}{0.41} = 2.278 \text{ bps}$$

Example 8: A discrete source emits one of five symbols once every millisecond. The symbol probabilities are 1/2, 1/4, 1/8, 1/16 and 1/16 respectively. Calculate the information rate.

Solution:

$$H = \sum_{i=1}^5 P_i \log_2 \frac{1}{P_i}$$

$$H = 0.5 + 0.5 + 0.375 + 0.25 + 0.25 = 1.875 \text{ bit/symbol}$$

$$R = \frac{H}{\tau} = \frac{1.875}{10^{-3}} = 1.875 \text{ kbps}$$

7.3. Mutual information for noisy channel

Consider the set of symbols $x_1, x_2, \dots, x_n$, the transmitter Tx may produce. The receiver Rx may receive $y_1, y_2, \dots, y_m$. Theoretically, if the noise and jamming is neglected, then the set $X = \text{set } Y$. However and due to noise and jamming, there will be a conditional probability $P(y_j | x_i)$:

1- $P(x_i)$ to be what is so called the a priori probability of the symbol x_i , which is the prob of selecting x_i for transmission.

2- $P(y_j | x_i)$ to be what is called the aposteriori probability of the symbol x_i after the reception of y_j .

The amount of information that y_j provides about x_i is called the mutual information between x_i and y_j . This is given by:

$$I(x_i, y_j) = \log_2 \left(\frac{\text{aposteriori prob}}{\text{apriori prob}} \right) = \log_2 \left(\frac{P(y_j | x_i)}{P(x_i)} \right)$$

Properties of $I(x_i, y_j)$:

- 1- It is symmetric, $I(x_i, y_j) = I(y_j, x_i)$.
- 2- $I(x_i, y_j) > 0$ if aposteriori probability > a priori probability, y_j provides +ve information about x_i .
- 3- $I(x_i, y_j) = 0$ if aposteriori probability = a priori probability, which is the case of statistical independence when y_j provides no information about x_i .
- 4- $I(x_i, y_j) < 0$ if aposteriori probability < a priori probability, y_j provides -ve information about x_i , or y_j adds ambiguity.

$$\text{Also } I(x_i, y_j) = \log_2 \left(\frac{P(x_i | y_j)}{P(y_j)} \right)$$

7.4. Joint entropy

In information theory, joint entropy is a measure of the uncertainty associated with a set of variables.

$$H(X, Y) = H(XY) = - \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2 P(x_i, y_j) \quad \frac{\text{bits}}{\text{symbol}}$$

7.5. Conditional entropy

In information theory, the conditional entropy quantifies the amount of information needed to describe the outcome of a random variable Y given that the value of another random variable X is known.

$$H(Y | X) = - \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2 P(y_j | x_i) \quad \text{bits/symbol}$$

7.6. Marginal Entropies

Marginal entropies is a term usually used to denote both source entropy $H(X)$ defined as before and the receiver entropy $H(Y)$ given by:

$$H(Y) = - \sum_{j=1}^m P(y_j) \log_2 P(y_j) \quad \frac{\text{bits}}{\text{symbol}}$$

7.7. Transinformation (average mutual information)

It is the statistical average of all pair $I(x_i, y_j)$, $i = 1, 2, \dots, n$, $j = 1, 2, \dots, m$. This is denoted by $I(X, Y)$ and is given by:

$$I(X, Y) = \sum_{i=1}^n \sum_{j=1}^m I(x_i, y_j) P(x_i, y_j)$$

$$I(X, Y) = \sum_{i=1}^n \sum_{j=1}^m P(x_i, y_j) \log_2 \left(\frac{P(y_j | x_i)}{P(y_j)} \right) \frac{\text{bits}}{\text{symbol}}$$

or

$$I(X, Y) = \sum_{i=1}^n \sum_{j=1}^m P(x_i, y_j) \log_2 \left(\frac{P(x_i | y_j)}{P(x_i)} \right) \text{bits/symbol}$$

7.8. Relationship between joint, conditional and transinformation

$$H(Y | X) = H(X, Y) - H(X)$$

$$H(X | Y) = H(X, Y) - H(Y)$$

Where, the $H(X | Y)$ is the losses entropy. Also we have:

$$I(X, Y) = H(X) - H(X | Y)$$

$$I(X, Y) = H(Y) - H(Y | X)$$

Example 9: The joint probability of a system is given by:

$$P(X, Y) = \begin{matrix} & \begin{matrix} x_1 & x_2 & x_3 \end{matrix} \\ \begin{matrix} y_1 \\ y_2 \\ y_3 \end{matrix} & \begin{bmatrix} 0.5 & 0.25 & 0 \\ 0 & 0.125 & 0.0625 \\ 0.0625 & 0.0625 & 0 \end{bmatrix} \end{matrix}$$

Find:

- 1- Marginal entropies.
- 2- Joint entropy
- 3- Conditional entropies.
- 4- The transinformation.

Solution:

$$1- P(X) = \begin{bmatrix} x_1 & x_2 & x_3 \\ 0.75 & 0.125 & 0.125 \end{bmatrix} \quad P(Y) = \begin{bmatrix} y_1 & y_2 \\ 0.5625 & 0.4375 \end{bmatrix}$$

$$H(X) = -[0.75 \ln(0.75) + 2 \times 0.125 \ln(0.125)]/\ln 2$$

$$= 1.06127 \text{ bits/symbol}$$

$$H(Y) = -[0.5625 \ln(0.5625) + 0.4375 \ln(0.4375)]/\ln 2$$

$$= 0.9887 \text{ bits/symbol}$$

2-

$$H(X, Y) = - \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2 P(x_i, y_j)$$

$$H(X, Y)$$

$$= - \frac{[0.5 \ln(0.5) + 0.25 \ln(0.25) + 0.125 \ln(0.125) + 2 \times 0.0625 \ln(0.0625)]}{\ln 2}$$

$$= 1.875 \text{ bits/symbol}$$

$$3- H(Y | X) = H(X, Y) - H(X) = 1.875 - 1.06127 = 0.813 \frac{\text{bits}}{\text{symbol}}$$

$$H(X | Y) = H(X, Y) - H(Y) = 1.875 - 0.9887 = 0.886 \text{ bits/symbol}$$

$$4- I(X, Y) = H(X) - H(X | Y) = 1.06127 - 0.8863 = 0.17497 \text{ bits/symbol.}$$

Chapter two

1. Channel

In telecommunications and computer networking, a communication channel refers either to a physical transmission medium, such as a wire, or to a logical connection over a multiplexed medium, such as a radio channel. A channel is used to convey an information signal, for example, a digital bit stream, from one or several senders (or transmitters) to one or several receivers. A channel has a certain capacity for transmitting information, often measured by its bandwidth in Hz or its data rate in bits per second.

2. Symmetric channel:

The symmetric channel has the following conditions:

a- Equal number of symbols in X&Y, i.e. $P(Y|X)$ is a square matrix.

b- Any row in the $P(Y|X)$ matrix comes from some permutation of the other rows.

For example, the following conditional probability of various channel types as shown:

a- $P(Y|X) = \begin{bmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{bmatrix}$ is a BSC, because it is square matrix and 1st row is the permutation of 2nd row.

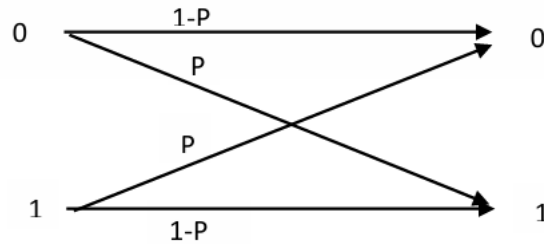
b- $P(Y|X) = \begin{bmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$ It is TSC because it is a square matrix, and each row is a permutation of the others.

c- $P(Y|X) = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 0.8 & 0.1 \end{bmatrix}$ is a non-symmetric since since it is not square although each row is permutation of others.

d- $P(Y|X) = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 0.7 & 0.2 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}$ is a non-symmetric although it is square since 2nd row is not permutation of other rows.

2.1.Binary symmetric channel (BSC)

It is a common communications channel model used in coding theory and information theory. In this model, a transmitter wishes to send a bit (a zero or a one), and the receiver receives a bit. It is assumed that the bit is usually transmitted correctly, but that it will be "flipped" with a small probability (the "crossover probability").



A binary symmetric channel with crossover probability p denoted by BSC_p , is a channel with binary input and binary output and probability of error p ; that is, if X is the transmitted random variable and Y the received variable, then the channel is characterized by the conditional probabilities:

$$Pr(Y = 0 \mid X = 0) = 1 - P$$

$$Pr(Y = 0 \mid X = 1) = P$$

$$Pr(Y = 1 \mid X = 0) = P$$

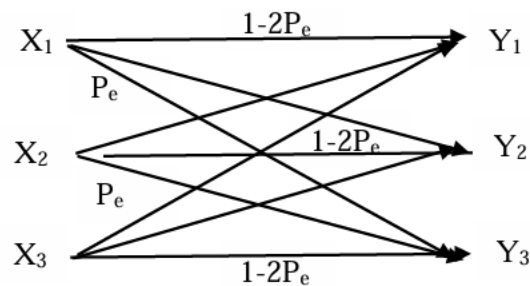
$$Pr(Y = 1 \mid X = 1) = 1 - P$$

2.2. Ternary symmetric channel (TSC)

The transitional probability of TSC is:

$$P(Y|X) = \begin{bmatrix} 1 - 2P_e & P_e & P_e \\ P_e & 1 - 2P_e & P_e \\ P_e & P_e & 1 - 2P_e \end{bmatrix}$$

The TSC is symmetric but not very practical, since in practice x_1 and x_3 are not affected as much as x_2 . In fact, the interference between x_1 and x_3 is much less than the interference between x_1 and x_2 or x_2 and x_3 .



Hence, the more practice, the more nonsymmetric the channel's transitional probability becomes.

$$P(Y|X) = \begin{bmatrix} 1 - P_e & P_e & 0 \\ P_e & 1 - 2P_e & P_e \\ 0 & P_e & 1 - P_e \end{bmatrix}$$

3. Shannon's theorem

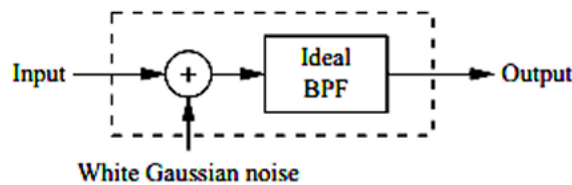
a- A given communication system has a maximum rate of information C known as the channel capacity.

b- If the information rate R is less than C , then one can approach arbitrarily small error probabilities by using intelligent coding techniques.

c- To get lower error probabilities, the encoder has to work on longer blocks of signal data. This entails longer delays and higher computational requirements.

Thus, if $R \leq C$ then transmission may be accomplished without error in the presence of noise. The negation of this theorem is also true: if $R > C$, then errors cannot be avoided regardless of the coding technique used.

Consider a bandlimited Gaussian channel operating in the presence of additive Gaussian noise:



The Shannon-Hartley theorem states that the channel capacity is given by:

$$C = B \log_2 \left(1 + \frac{S}{N} \right)$$

Where C is the capacity in bits per second, B is the bandwidth of the channel in Hertz, and S/N is the signal-to-noise ratio.

Example 1: A channel having a bandwidth of 1 kHz, the power of signal is 12 W and the power of noise is 4 W. Determine the channel capacity.

Solution:

$$C = B \log_2 \left(1 + \frac{S}{N} \right) = 1 \times 10^3 \log_2 \left(1 + \frac{12}{4} \right) = 2 \text{ bps.}$$

Or: A channel having a bandwidth of 1 kHz and Signal to Noise Ratio (SNR) is 3. Determine the channel capacity.

Example 2: If the channel capacity is 2000 bps, find its bandwidth if the SNR is 3.

Solution:

$$C = B \log_2 \left(1 + \frac{S}{N} \right)$$

$$2000 = B \log_2(1 + 3) = B \log_2(4)$$

$$2000 = B \times 2 \rightarrow B = 1000 \text{ kHz}$$

Example 3: Determine the SNR required to achieve a channel capacity of 1585 bps, if the bandwidth is 1 kHz.

Solution:

$$C = B \log_2(1 + \text{SNR})$$

$$1585 = 1000 \log_2(1 + \text{SNR})$$

$$\frac{1585}{1000} = \log_2(1 + \text{SNR})$$

$$1.585 = \log_2(1 + \text{SNR})$$

$$\log_2^{-1}(1.585) = (1 + \text{SNR})$$

$$3 = (1 + \text{SNR})$$

$$\text{SNR} = 3 - 1 = 2$$

4. Channel Capacity (Discrete channel)

This is defined as the maximum of $I(X, Y)$:

$$C = \text{channel capacity} = \max[I(x, y)] \quad \frac{\text{bits}}{\text{symbol}}$$

Physically it is the maximum amount of information each symbol can carry to the receiver. Sometimes this capacity is also expressed in bits/sec if related to the rate of producing symbols r :

$$R(X, Y) = r \times I(X, Y) \text{ bits/sec or } R(X, Y) = I(X, Y) / \bar{\tau}$$

Channel capacity of Symmetric channels:

The channel capacity is defined as $\max [I(X, Y)]$

$$I(X, Y) = H(Y) - H(Y | X)$$

$$I(X, Y) = H(Y) + \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2(y_j | x_i)$$

But we have

$$P(x_i, y_j) = P(x_i)P(y_j | x_i) \quad \text{put in above equation yields:}$$

$$I(X, Y) = H(Y) + \sum_{j=1}^m \sum_{i=1}^n P(x_i)P(y_j | x_i) \log_2 P(y_j | x_i)$$

If the channel is symmetric the quantity:

$$\sum_{j=1}^m P(y_j | x_i) \log_2 P(y_j | x_i) = K$$

Where K is constant and independent of the row number i so that the equation becomes:

$$I(X, Y) = H(Y) + K \sum_{i=1}^n P(x_i)$$

Hence $I(X, Y) = H(Y) + K$ for symmetric channels

$$\text{Max of } I(X, Y) = \max[H(Y) + K] = \max[H(Y)] + K$$

When Y has equiprobable symbols then $\max[H(Y)] = \log_2 m$

Then

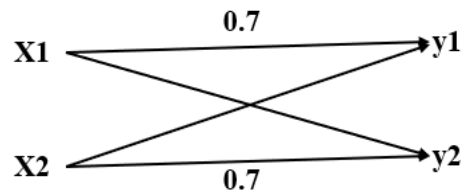
Then

$$5. I(X, Y) = \log_2 m + K$$

Or

$$C = \log_2 m + K$$

Example 4: For the BSC shown below:



Find the channel capacity and efficiency if $I(x_1) = 2 \text{ bits}$.

Solution:

$$P(Y | X) = \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix}$$

Since the channel is symmetric then

$$C = \log_2 m + K \quad \text{and } n = m$$

where n and m are number row and column respectively

$$K = 0.7 \log_2 0.7 + 0.3 \log_2 0.3 = -0.88129$$

$$C = 1 - 0.88129 = 0.1187 \text{ bits/symbol}$$

The channel efficiency $\eta = \frac{I(X,Y)}{C}$

$$I(x_1) = -\log_2 P(x_1) = 2$$

$$P(x_1) = 2^{-2} = 0.25 \quad \text{then } P(X) = [0.25 \quad 0.75]^T$$

And we have $P(x_i, y_j) = P(x_i)P(y_j | x_i)$ so that

$$P(X, Y) = \begin{bmatrix} 0.7 \times 0.25 & 0.3 \times 0.25 \\ 0.3 \times 0.75 & 0.7 \times 0.75 \end{bmatrix} = \begin{bmatrix} 0.175 & 0.075 \\ 0.225 & 0.525 \end{bmatrix}$$

$$I(Y) = [0.4 \ 0.6] \rightarrow H(Y) = 0.97095 \frac{\text{bits}}{\text{symbol}}$$

$$I(X, Y) = H(Y) + K = 0.97095 - 0.88129 = 0.0896 \text{ bits/symbol}$$

$$\text{Then } \eta = \frac{0.0896}{0.1187} = 75.6\%$$

To find the channel redundancy:

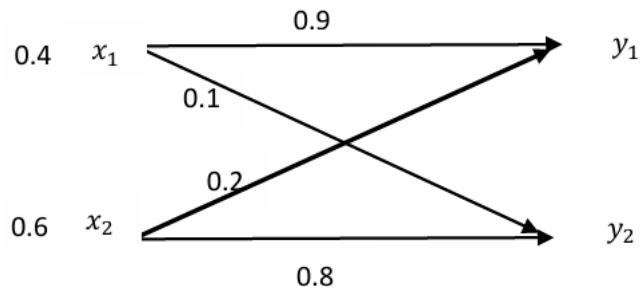
$$R = 1 - \eta = 1 - 0.756 = 0.244 \text{ or } 24.4\%$$

Example 5: A binary source sending x_1 with a probability of 0.4 and x_2 with 0.6 probability through a channel with a probabilities of errors of 0.1 for x_1 and 0.2 for x_2 . Determine:

- 1- Source entropy.
- 2- Marginal entropy.
- 3- Joint entropy.
- 4- Conditional entropy $H(Y | X)$.
- 5- Losses entropy $H(X | Y)$.
- 6- Transinformation.

Solution:

1- The channel diagram:



$$\text{Or } P(Y|X) = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

$$H(X) = - \frac{[0.4 \ln(0.4) + 0.6 \ln(0.6)]}{\ln 2} = 0.971 \frac{\text{bits}}{\text{symbol}}$$

$$2- P(X,Y) = P(Y/X) \times P(X)$$

$$P(Y|X) = \begin{bmatrix} 0.9 \times 0.4 & 0.1 \times 0.4 \\ 0.2 \times 0.6 & 0.8 \times 0.6 \end{bmatrix} = \begin{bmatrix} 0.36 & 0.04 \\ 0.12 & 0.48 \end{bmatrix}$$

$$H(Y) = - \sum_{j=1}^m P(y_j) \log_2 P(y_j) \quad \frac{\text{bits}}{\text{symbol}}$$

$$H(Y) = 0.999 \frac{\text{bits}}{\text{symbols}}$$

$$3- H(X,Y)$$

$$H(X,Y) = - \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2 P(x_i, y_j)$$

$$H(X,Y) = 1.592 \text{ bits/symbol}$$

$$4- H(Y|X)$$

$$\begin{aligned}
 H(Y | X) &= - \sum_{j=1}^m \sum_{i=1}^n P(x_i, y_j) \log_2 P(y_j | x_i) \quad \text{bits/symbol} \\
 &= - \frac{[0.36 \ln(0.9) + 0.12 \ln(0.2) + 0.04 \ln(0.1) + 0.48 \ln(0.8)]}{\ln(2)} \\
 &= 0.621 \frac{\text{bits}}{\text{symbol}}
 \end{aligned}$$

$$\text{Or } H(Y | X) = H(X, Y) - H(X) = 1.592 - 0.971 = 0.621 \frac{\text{bits}}{\text{symbol}}$$

$$5- H(X | Y) = H(X, Y) - H(Y) = 1.592 - 0.999 = 0.593 \text{ bits/symbol}$$

$$6- I(X, Y) = H(X) - H(X | Y) = 0.971 - 0.593 = 0.378 \text{ bits/symbol}$$

6. Cascading of Channels

If two channels are cascaded, then the overall transition matrix is the product of the two transition matrices.

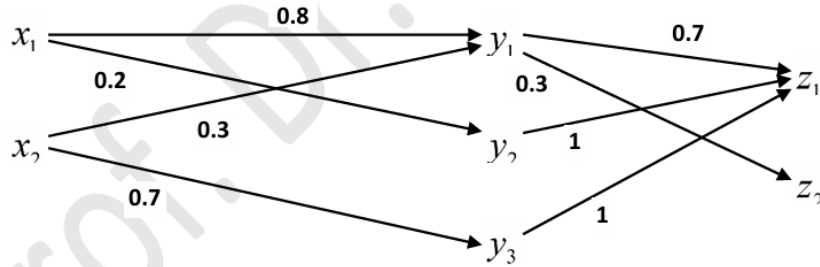
$$p(z/x) = p(y/x) \cdot p(z/y)$$

$$\begin{matrix} (n \times k) & (n \times m) & (m \times k) \\ \text{matrix} & \text{matrix} & \text{matrix} \end{matrix}$$

For the series information channel, the overall channel capacity does not exceed any of the channels individually.

$$I(X, Z) \leq I(X, Y) \text{ \& } I(X, Z) \leq I(Y, Z)$$

Example 6: Find the transition matrix for the cascaded channel shown.



Solution:

$$p(y/x) = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0 & 0.7 \end{bmatrix}, \quad p(z/y) = \begin{bmatrix} 0.7 & 0.3 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$p(z/x) = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0 & 0.7 \end{bmatrix} \begin{bmatrix} 0.7 & 0.3 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0.76 & 0.24 \\ 0.91 & 0.09 \end{bmatrix}$$

Chapter Three

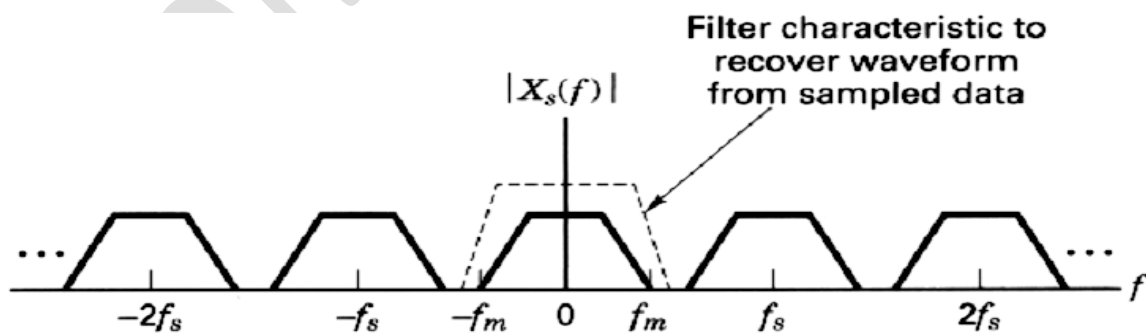
Source Coding

1- Sampling theorem:

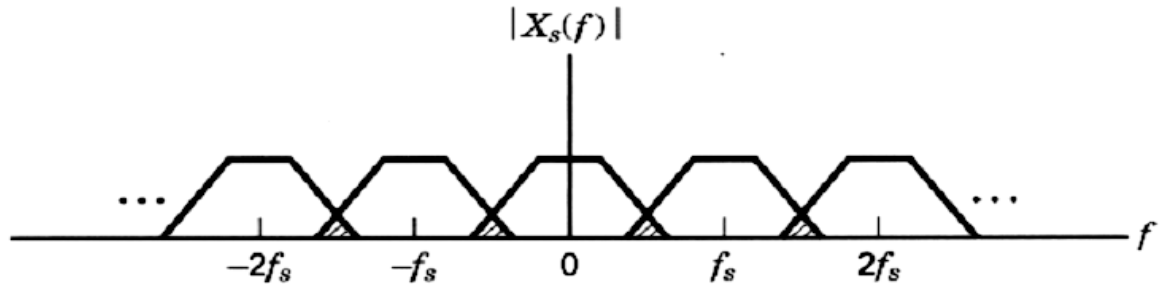
Sampling of the signals is the fundamental operation in digital communication. A continuous time signal is first converted to discrete time signal by sampling process. Also it should be possible to recover or reconstruct the signal completely from its samples.

When the sampling rate is chosen $f_s = 2f_m$ each spectral replicate is separated from each of its neighbors by a frequency band exactly equal to f_s hertz, and the analog waveform can theoretically be completely recovered from the samples, by the use of filtering. It should be clear that if $f_s > 2f_m$, the replications will be move farther apart in frequency making it easier to perform the filtering operation.

When the sampling rate is reduced, such that $f_s < 2f_m$, the replications will overlap, as shown in figure below, and some information will be lost. This phenomenon is called aliasing.



Sampled spectrum $f_s > 2f_m$



Sampled spectrum $f_s < 2f_m$

A bandlimited signal having no spectral components above f_m hertz can be determined uniquely by values sampled at uniform intervals of $T_s \leq \frac{1}{2f_m}$ sec.

The sampling rate is $f_s = \frac{1}{T_s}$

So that $f_s \geq 2f_m$. The sampling rate $f_s = 2f_m$ is called Nyquist rate.

Example: Find the Nyquist rate and Nyquist interval for the following signals.

i- $m(t) = \frac{\sin(500\pi t)}{\pi t}$

ii- $m(t) = \frac{1}{2\pi} \cos(4000\pi t) \cos(1000\pi t)$

Solution:

i- $\omega t = 500\pi t \quad \therefore 2\pi f = 500\pi \quad \rightarrow f = 250\text{Hz}$

$$\text{Nyquist interval} = \frac{1}{2f_{\max}} = \frac{1}{2 \times 250} = 2 \text{ msec.}$$

$$\text{Nyquist rate} = 2f_{\max} = 2 \times 250 = 500\text{Hz}$$

$$\begin{aligned} \text{ii- } m(t) &= \frac{1}{2\pi} \left[\frac{1}{2} \{ \cos(4000\pi t - 1000\pi t) + \cos(4000\pi t + 1000\pi t) \} \right] \\ &= \frac{1}{4\pi} \{ \cos(3000\pi t) + \cos(5000\pi t) \} \end{aligned}$$

Then the highest frequency is 2500Hz

$$\text{Nyquist interval} = \frac{1}{2f_{max}} = \frac{1}{2 \times 2500} = 0.2 \text{ msec.}$$

$$\text{Nyquist rate} = 2f_{max} = 2 \times 2500 = 5000 \text{ Hz}$$

H. W:

Find the Nyquist interval and Nyquist rate for the following:

i- $\frac{1}{2\pi} \cos(400\pi t) \cdot \cos(200\pi t)$

ii- $\frac{1}{\pi} \sin \pi t$

Example:

A waveform $[20 + 20\sin(500t + 30^\circ)]$ is to be sampled periodically and reproduced from these sample values. Find maximum allowable time interval between sample values, how many sample values are needed to be stored in order to reproduce 1 sec of this waveform?.

Solution:

$$x(t) = 20 + 20 \sin(500t + 30^\circ)$$

$$\omega = 500 \rightarrow 2\pi f = 500 \rightarrow f = 79.58 \text{ Hz}$$

Minimum sampling rate will be twice of the signal frequency:

$$f_{s(\min)} = 2 \times 79.58 = 159.15 \text{ Hz}$$

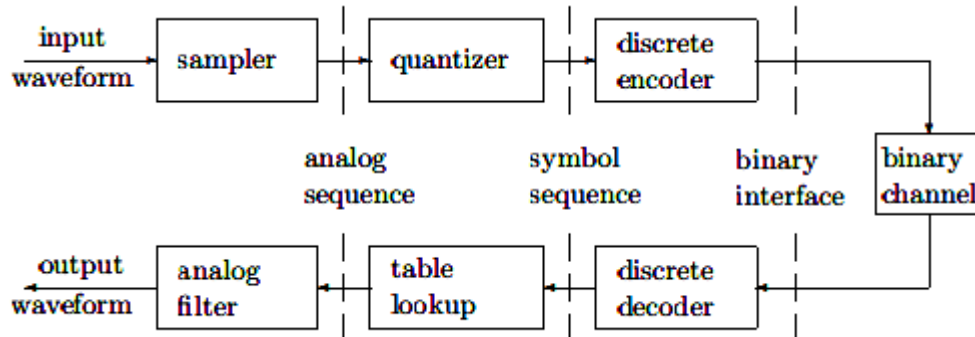
$$T_{s(\max)} = \frac{1}{f_{s(\min)}} = \frac{1}{159.15} = 6.283 \text{ msec.}$$

$$\text{Number of sample in 1sec} = \frac{1}{6.283 \text{ msec}} = 159.16 \approx 160 \text{ sample}$$

2- Source coding:

An important problem in communications is the efficient representation of data generated by a discrete source. The process by which this representation is accomplished is called source encoding. An efficient source encoder must satisfies two functional requirements:

- i- The code words produced by the encoder are in binary form.
- ii- The source code is uniquely decodable, so that the original source sequence can be reconstructed perfectly from the encoded binary sequence.



The entropy for a source with statistically independent symbols:

$$H(Y) = - \sum_{j=1}^m P(y_j) \log_2 P(y_j)$$

We have:

$$\max[H(Y)] = \log_2 m$$

A code efficiency can therefore be defined as:

$$\eta = \frac{H(Y)}{\max[H(Y)]} \times 100$$

The overall code length, L , can be defined as the average code word length:

$$L = \sum_{j=1}^m P(x_j) l_j \quad \text{bits/symbol}$$

The code efficiency can be found by:

$$\eta = \frac{H(Y)}{L} \times 100$$

Not that $\max[H(Y)] \quad \text{bits/symbol} = L \quad \text{bits/codeword}$

i- Fixed- Length Code Words:

If the alphabet X consists of the 7 symbols $\{a, b, c, d, e, f, g\}$, then the following fixed-length code of block length $L = 3$ could be used.

$C(a) = 000$
 $C(b) = 001$
 $C(c) = 010$
 $C(d) = 011$
 $C(e) = 100$
 $C(f) = 101$
 $C(g) = 110.$

The encoded output contains L bits per source symbol. For the above example the source sequence *bad...* would be encoded into 001000011... . Note that the output bits are simply run together (or, more technically, concatenated). This method is nonprobabilistic; it takes no account of whether some symbols occur more frequently than others, and it works robustly regardless of the symbol frequencies.

This is used when the source produces almost equiprobable messages $p(x_1) \cong p(x_2) \cong p(x_3) \cong \dots \cong p(x_n)$, then $l_1 = l_2 = l_3 = \dots = l_n = L_C$ and for binary coding then:

- 1- $L_C = \log_2 n$ bit/message if $n = 2^r$ ($n = 2, 4, 8, 16, \dots$ and r is an integer) which gives $\eta = 100\%$
- 2- $L_C = \text{Int}[\log_2 n] + 1$ bits/message if $n \neq 2^r$ which gives less efficiency

Example

For ten equiprobable messages coded in a fixed length code then

$$p(x_i) = \frac{1}{10} \text{ and } L_C = \text{Int}[\log_2 10] + 1 = 4 \text{ bits}$$

$$\text{and } \eta = \frac{H(X)}{L_C} \times 100\% = \frac{\log_2 10}{4} \times 100\% = 83.048\%$$

Example: For eight equiprobable messages coded in a fixed length code then

$$p(x_i) = \frac{1}{8} \text{ and } L_C = \log_2 8 = 3 \text{ bits and } \eta = \frac{3}{3} \times 100\% = 100\%$$

Example: Find the efficiency of a fixed length code used to encode messages obtained from throwing a fair die (a) once, (b) twice, (c) 3 times.

Solution

- a- For a fair die, the messages obtained from it are equiprobable with a probability of $p(x_i) = \frac{1}{6}$ with $n = 6$.

$$L_C = \text{Int}[\log_2 6] + 1 = 3 \text{ bits/message}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = \frac{\log_2 6}{3} \times 100\% = 86.165\%$$

- b- For two throws then the possible messages are $n = 6 \times 6 = 36$ messages with equal probabilities

$$L_C = \text{Int}[\log_2 36] + 1 = 6 \text{ bits/message} = 6 \text{ bits/2-symbols}$$

while $H(X) = \log_2 6$ bits/symbol $\eta = \frac{2 \times H(X)}{L_C} \times 100\% = 86.165\%$

- c- For three throws then the possible messages are $n = 6 \times 6 \times 6 = 216$ with equal probabilities

$$L_C = \text{Int}[\log_2 216] + 1 = 8 \text{ bits/message} = 8 \text{ bits/3-symbols}$$

while $H(X) = \log_2 6$ bits/symbol $\eta = \frac{3 \times H(X)}{L_C} \times 100\% = 96.936\%$

ii- Variable-Length Code Words

When the source symbols are not equally probable, a more efficient encoding method is to use variable-length code words. For example, a variable-length code for the alphabet $X = \{a, b, c\}$ and its lengths might be given by

$$C(a) = 0 \quad l(a) = 1$$

$$C(b) = 10 \quad l(b) = 2$$

$$C(c) = 11 \quad l(c) = 2$$

The major property that is usually required from any variable-length code is that of unique decodability. For example, the above code C for the alphabet $X = \{a, b, c\}$ is soon shown to be uniquely decodable. However such code is not uniquely decodable, even though the codewords are all different. If the source decoder observes 01, it cannot determine whether the source emitted (a b) or (c).

Prefix-free codes: A **prefix code** is a type of code system (typically a variable-length code) distinguished by its possession of the "prefix property", which requires that there is no code word in the system that is a prefix (initial segment) of any other code word in the system. For example:

$\{a = 0, b = 110, c = 10, d = 111\}$ is a prefix code.

When message probabilities are not equal, then we use variable length codes. The following properties need to be considered when attempting to use variable length codes:

1) Unique decoding:

Example

Consider a 4 alphabet symbols with symbols represented by binary digits as follows:

$A = 0$

$B = 01$

$C = 11$

$D = 00$

If we receive the code word 0011 it is not known whether the transmission was DC or AAC . This example is not, therefore, uniquely decodable.

2) Instantaneous decoding:

Example

Consider a 4 alphabet symbols with symbols represented by binary digits as follows:

$A = 0$

$$B = 10$$

$$C = 110$$

$$D = 111$$

This code can be instantaneously decoded since no complete codeword is a prefix of a larger codeword. This is in contrast to the previous example where A is a prefix of both B and D . This example is also a ‘comma code’ as the symbol zero indicates the end of a codeword except for the all ones word whose length is known.

Example

Consider a 4 alphabet symbols with symbols represented by binary digits as follows:

$$A = 0$$

$$B = 01$$

$$C = 011$$

$$D = 111$$

The code is identical to the previous example but the bits are time reversed. It is still uniquely decodable but no longer instantaneous, since early codewords are now prefixes of later ones.

Shannon Code

For messages $x_1, x_2, x_3, \dots, x_n$ with probabilities $p(x_1), p(x_2), p(x_3), \dots, p(x_n)$ then:

$$1) l_i = -\log_2 p(x_i) \quad \text{if } p(x_i) = \left(\frac{1}{2}\right)^r \quad \left\{\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots\right\}$$

$$2) l_i = \text{Int}[-\log_2 p(x_i)] + 1 \quad \text{if } p(x_i) \neq \left(\frac{1}{2}\right)^r$$

$$\text{Also define} \quad F_i = \sum_{k=1}^{i-1} p(x_k) \quad 1 \geq \omega_i \geq 0$$

then the codeword of x_i is the binary equivalent of F_i consisting of l_i bits.

$$C_i = (F_i)_2^{l_i}$$

where C_i is the binary equivalent of F_i up to l_i bits. In encoding, messages must be arranged in a decreasing order of probabilities.

Example

Develop the Shannon code for the following set of messages,

$$p(x) = [0.3 \ 0.2 \ 0.15 \ 0.12 \ 0.1 \ 0.08 \ 0.05]$$

then find:

- (a) Code efficiency,
 (b) $p(0)$ at the encoder output.

Solution

x_i	$p(x_i)$	l_i	F_i	C_i	0_i
x_1	0.3	2	0	00	2
x_2	0.2	3	0.3	010	2
x_3	0.15	3	0.5	100	2
x_4	0.12	4	0.65	1010	2
x_5	0.10	4	0.77	1100	2
x_6	0.08	4	0.87	1101	1
x_7	0.05	5	0.95	11110	1

To find C_1

$$0 \times 2 = 0 \quad 0 \downarrow$$

To find C_2

$$\begin{array}{l} 0.3 \times 2 = 0.6 \quad 0 \downarrow \\ 0.6 \times 2 = 1.2 \quad 1 \downarrow \end{array}$$

To find C_3

$$\begin{array}{l} 0.5 \times 2 = 1.0 \quad 1 \downarrow \\ 0.0 \times 2 = 0.0 \quad 0 \downarrow \end{array}$$

To find C_4

$$\begin{array}{l} 0.65 \times 2 = 1.3 \quad 1 \downarrow \\ 0.30 \times 2 = 0.6 \quad 0 \downarrow \\ 0.60 \times 2 = 1.2 \quad 1 \downarrow \\ 0.20 \times 2 = 0.4 \quad 0 \downarrow \end{array}$$

To find C_5

$$\begin{array}{l} 0.77 \times 2 = 1.54 \quad 1 \downarrow \\ 0.54 \times 2 = 1.08 \quad 1 \downarrow \\ 0.08 \times 2 = 0.16 \quad 0 \downarrow \\ 0.16 \times 2 = 0.32 \quad 0 \downarrow \end{array}$$

(a) To find the code efficiency, we have

$$L_C = \sum_{i=1}^7 l_i p(x_i) = 3.1 \text{ bits/message.}$$

$$H(X) = -\sum_{i=1}^7 p(x_i) \log_2 p(x_i) = 2.6029 \text{ bits/message.}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = 83.965\%$$

(b) $p(0)$ at the encoder output is

$$p(0) = \frac{\sum_{i=1}^7 0_i p(x_i)}{L_C} = \frac{0.6 + 0.4 + 0.3 + 0.24 + 0.2 + 0.08 + 0.05}{3.1}$$

$$p(0) = 0.603$$

Example

Repeat the previous example using ternary coding.

Solution

$$1) l_i = -\log_3 p(x_i) \quad \text{if } p(x_i) = \left(\frac{1}{3}\right)^r \quad \left\{\frac{1}{3}, \frac{1}{9}, \frac{1}{27}, \dots\right\}$$

$$2) l_i = \text{Int}[-\log_3 p(x_i)] + 1 \quad \text{if } p(x_i) \neq \left(\frac{1}{3}\right)^r \quad \text{and} \quad C_i = (F_i)_3^{l_i}$$

x_i	$p(x_i)$	l_i	F_i	C_i	0_i
x_1	0.3	2	0	00	2
x_2	0.2	2	0.3	02	1
x_3	0.15	2	0.5	11	0
x_4	0.12	2	0.65	12	0
x_5	0.10	3	0.77	202	1
x_6	0.08	3	0.87	212	0

x_7	0.05	3	0.95	221	0
-------	------	---	------	-----	---

To find C_1

$$0 \times 3 = 0 \quad 0 \downarrow$$

To find C_2

$$0.3 \times 3 = 0.9 \quad 0 \downarrow$$

To find C_3

$$0.5 \times 3 = 1.5 \quad 1 \downarrow$$

To find C_4

$$0.65 \times 3 = 1.95 \quad 1 \downarrow$$

$$0.95 \times 3 = 2.85 \quad 2$$

To find C_5

$$0.77 \times 3 = 2.31 \quad 2 \downarrow$$

$$0.31 \times 3 = 0.93 \quad 0$$

(a) To find the code efficiency, we have

$$L_C = \sum_{i=1}^7 l_i p(x_i) = 2.23 \text{ ternary unit/message.}$$

$$H(X) = -\sum_{i=1}^7 p(x_i) \log_3 p(x_i) = 1.642 \text{ ternary unit/message.}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = 73.632\%$$

(b) $p(0)$ at the encoder output is

$$p(0) = \frac{\sum_{i=1}^7 0_i p(x_i)}{L_C} = \frac{0.6 + 0.2 + 0.1}{2.23}$$

$$p(0) = 0.404$$

Shannon- Fano Code:

In Shannon–Fano coding, the symbols are arranged in order from most probable to least probable, and then divided into two sets whose total probabilities are as close as possible to being equal. All symbols then have the first digits of their codes assigned; symbols in the first set receive "0" and symbols in the second set receive "1". As long as any sets with more than one member remain, the same process is repeated on those sets, to determine successive digits of their codes.

Example:

The five symbols which have the following frequency and probabilities, design suitable Shannon-Fano binary code. Calculate average code length, source entropy and efficiency.

Symbol	count	Probabilities	Binary codes	Length
A	15	0.385	00	2
B	7	0.1795	01	2
C	6	0.154	10	2
D	6	0.154	110	3
E	5	0.128	111	3

The average code word length:

$$L = \sum_{j=1}^m P(x_j) l_j$$

$$L = 2 \times 0.385 + 2 \times 0.1793 + 2 \times 0.154 + 3 \times 0.154 + 3 \times 0.128$$

$$= 2.28 \text{ bits/symbol}$$

The source entropy is:

$$H(Y) = - \sum_{j=1}^m P(y_j) \log_2 P(y_j)$$

$$H(Y) = -[0.385 \ln 0.385 + 0.1793 \ln 0.1793 + 2 \times 0.154 \ln 0.154 + 0.128 \ln 0.128] / \ln 2$$

$$H(Y) = 2.18567 \text{ bits/symbol}$$

The code efficiency:

$$\eta = \frac{H(Y)}{L} \times 100 = \frac{2.18567}{2.28} \times 100 = 95.86\%$$

Example

Develop the Shannon - Fano code for the following set of messages, $p(x)=[0.35 \ 0.2 \ 0.15 \ 0.12 \ 0.1 \ 0.08]$ then find the code efficiency.

Solution

x_i	$p(x_i)$	Code			l_i
x_1	0.35	0	0		2
x_2	0.2	0	1		2
x_3	0.15	1	0	0	3
x_4	0.12	1	0	1	3
x_5	0.10	1	1	0	3
x_6	0.08	1	1	1	3

$$L_C = \sum_{i=1}^6 l_i p(x_i) = 2.45 \text{ bits/symbol}$$

$$H(X) = -\sum_{i=1}^6 p(x_i) \log_2 p(x_i) = 2.396 \text{ bits/symbol}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = 97.796\%$$

Example

Repeat the previous example using with $r = 3$

Solution

x_i	$p(x_i)$	Code		l_i
x_1	0.35	0		1
x_2	0.2	1	0	2
x_3	0.15	1	1	2

x_4	0.12	2	0	2
x_5	0.10	2	1	2
x_6	0.08	2	2	2

$$L_C = \sum_{i=1}^6 l_i p(x_i) = 1.65 \quad \text{ternary unit/symbol}$$

$$H(X) = -\sum_{i=1}^6 p(x_i) \log_3 p(x_i) = 1.512 \quad \text{ternary unit/symbol}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = 91.636\%$$

Huffman Code

The Huffman coding algorithm comprises two steps, reduction and splitting. These steps can be summarized as follows:

1) Reduction

- List the symbols in descending order of probability.
- Reduce the r least probable symbols to one symbol with a probability equal to their combined probability.
- Reorder in descending order of probability at each stage.
- Repeat the reduction step until only two symbols remain.

2) Splitting

- Assign $0, 1, \dots, r$ to the r final symbols and work backwards.

b) Expand or lengthen the code to cope with each successive split.

Example: Design Huffman codes for $A = \{a_1, a_2, \dots, a_5\}$, having the probabilities $\{0.2, 0.4, 0.2, 0.1, 0.1\}$.

Symbol	Step 1	Step 2	Step 3	Step 4	Codeword
a_2	0.4	0.4	0.4	0.6 0	1
a_1	0.2	0.2	0.4	0.4 1	01
a_3	0.2	0.2	0.2		000
a_4	0.1	0.2			0010
a_5	0.1				0011

Letter	Probability	Codeword
a_2	0.4	1
a_1	0.2	01
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

The average code word length:

$$L = 0.4 \times 1 + 0.2 \times 2 + 0.2 \times 3 + 0.1 \times 4 + 0.1 \times 4 = 2.2 \text{ bits/symbol}$$

The source entropy:

$$H(Y) = -[0.4 \ln 0.4 + 2 \times 0.2 \ln 0.2 + 2 \times 0.1 \ln 0.1] / \ln 2 = 2.12193 \text{ bits/symbol}$$

The code efficiency:

$$\eta = \frac{2.12193}{2.2} \times 100 = 96.45\%$$

It can be design Huffman codes with minimum variance:

Symbol	Step 1	Step 2	Step 3	Step 4	Codeword
a_2	0.4	0.4	0.4	0.6 0	00
a_1	0.2	0.2	0.4 } 0	0.4 1	10
a_3	0.2	0.2 } 0	0.2 } 1		11
a_4	0.1 } 0	0.2 } 1			010
a_5	0.1 } 1				011

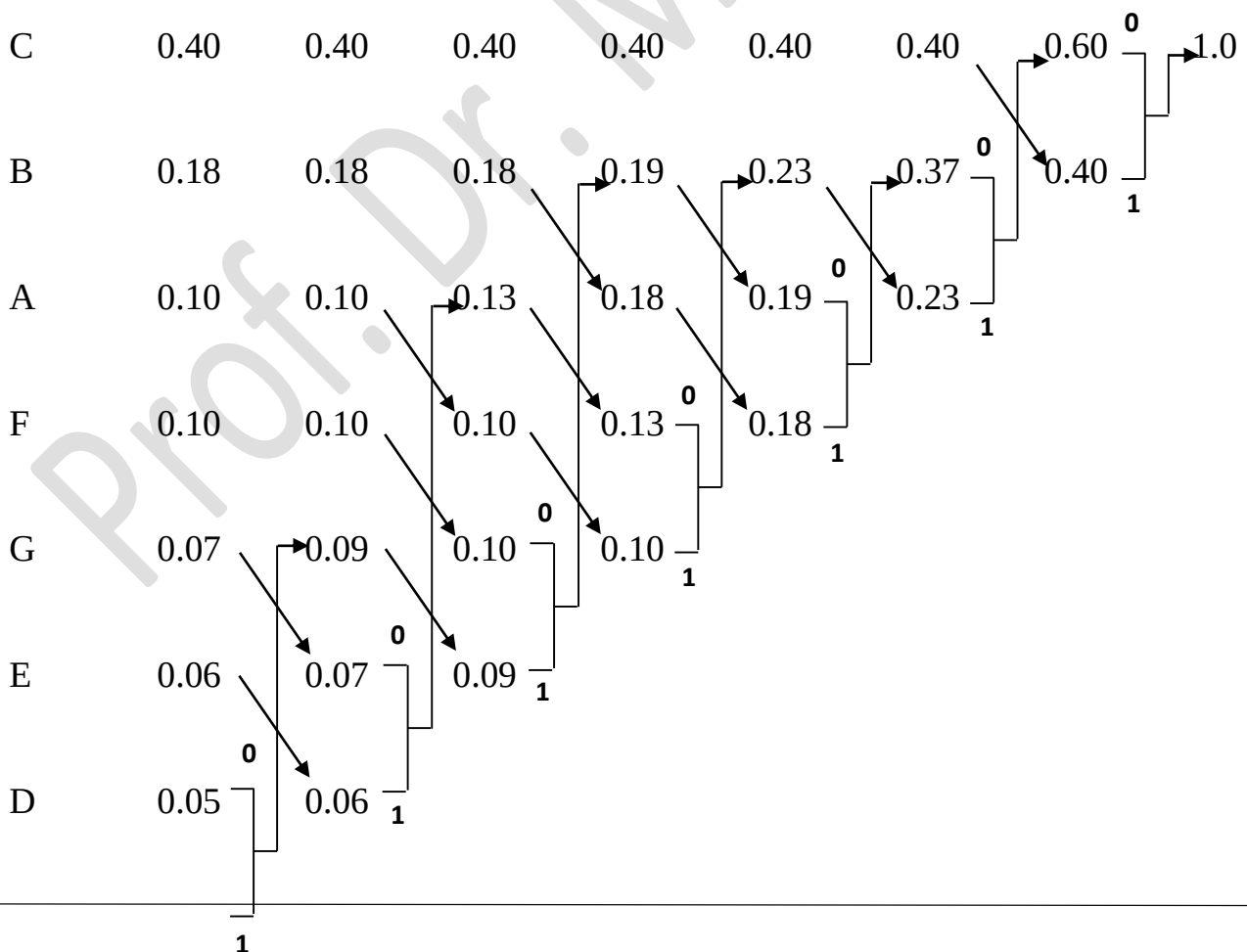
The average code word length is still 2.2 bits/symbol. But variances are different!

Example

Develop the Huffman code for the following set of symbols

Symbol	A	B	C	D	E	F	G	H
Probability	0.1	0.18	0.4	0.05	0.06	0.1	0.07	0.04

Solution



H 0.04

So we obtain the following codes

Symbol	A	B	C	D	E	F	G	H
Probability	0.1	0.18	0.4	0.05	0.06	0.1	0.07	0.04
Codeword	011	001	1	00010	0101	0000	0100	00011
l_i	3	3	1	5	4	4	4	5

$$H(X) = -\sum_{i=1}^8 p(x_i) \log_2 p(x_i) = 2.552 \text{ bits/symbol}$$

$$L_C = \sum_{i=1}^8 l_i p(x_i) = 2.61 \text{ bits/symbol}$$

$$\eta = \frac{H(X)}{L_C} \times 100\% = 97.778\%$$

Data Compression:

In computer science and information theory, data compression, source coding, or bit-rate reduction involves encoding information using fewer bits than the original representation. Compression can be either lossy or lossless.

Lossless data compression algorithms usually exploit statistical redundancy to represent data more concisely without losing information, so that the process is reversible. Lossless compression is possible because most real-world data has statistical redundancy. For example, an image may have areas of color that do not change over several pixels.

Lossy data compression is the converse of lossless data compression. In these schemes, some loss of information is acceptable. Dropping nonessential detail from the data source can save storage space. There is a corresponding trade-off between preserving information and reducing size.

Run-Length Encoding (RLE):

Run-Length Encoding is a very simple lossless data compression technique that replaces runs of two or more of the same character with a number which represents the length of the run, followed by the original character; single characters are coded as runs of 1. RLE is useful for highly-redundant data, indexed images with many pixels of the same color in a row.

Example:

Input: AAABBCCCCDEEEEEAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAA
Output: 3A2B4C1D6E38A

The input message to RLE encoder is a variable while the output code word is fixed, unlike Huffman code where the input is fixed while the output is varied.

Example : Consider these repeated pixels values in an image ... 0 0 0 0 0 0 0 0 0 0 0 5 5 5 5 0 0 0 0 0 0 0 We could represent them more efficiently as (12, 0)(4, 5)(8, 0) 24 bytes reduced to 6 which gives a compression ratio of $24/6 = 4:1$.

Example :Original Sequence (1 Row): 11112223333311112222 can be encoded as: (4,1),(3,2),(6,3),(4,1),(4,2). 21 bytes reduced to 10 gives a compression ratio of $21/10 = 2.1:1$.

Example : Original Sequence (1 Row): – HHHHHHHUFFFFFFFFFFFFFFFFF can be encoded as: (7,H),(1,U),(14,F) . 22 bytes reduced to 6 gives a compression ratio of $22/6 = 11:3$.

Savings Ratio : the savings ratio is related to the compression ratio and is a measure of the amount of redundancy between two representations (compressed and uncompressed). Let:

N_1 = the total number of bytes required to store an uncompressed (raw) source image.

N_2 = the total number of bytes required to store the compressed data.

The compression ratio C_r is then defined as:

$$C_r = \frac{N_1}{N_2}$$

- ▶ Larger compression ratios indicate more effective compression
- ▶ Smaller compression ratios indicate less effective compression
- ▶ Compression ratios less than one indicate that the uncompressed representation has high degree of irregularity.

The saving ratio S_r is then defined as :

$$S_r = \frac{(N_1 - N_2)}{N_1}$$

- ▶ Higher saving ratio indicate more effective compression while negative ratios are possible and indicate that the compressed image has larger memory size than the original.

Example: a 5 Megabyte image is compressed into a 1 Megabyte image, the savings ratio is defined as $(5-1)/5$ or $4/5$ or 80%.

This ratio indicates that 80% of the uncompressed data has been eliminated in the compressed encoding.

Chapter Four

1. Error detection and correction codes

The idea of error detection and/or correction is to add extra bits to the digital message that enable the receiver to detect or correct errors with limited capabilities. These extra bits are called parity bits. If we have k bits, r parity bits are added, then the transmitted digits are:

$$n = r + k$$

Here n called code word denoted as (n, k) . The efficiency or code rate is equal to k/n .

Two basic approaches to error correction are available, which are:

a- Automatic-repeat-request (ARQ): Discard those frames in which errors are detected. - For frames in which no error was detected, the receiver returns a positive acknowledgment to the sender. - For the frame in which errors have been detected, the receiver returns negative acknowledgement to the sender.

b- Forward error correction (FEC): Ideally, FEC codes can be used to generate encoding symbols that are transmitted in packets in such a way that each received packet is fully useful to a receiver to reassemble the object regardless of previous packet reception patterns. The most applications of FEC are: Compact Disc (CD) applications, digital audio and video, Global System Mobile (GSM) and Mobile communications.

2. Basic definitions :

- Systematic and nonsystematic codes: If $(a$'s) information bits are unchanged in their values and positions at the transmitted code word, then this code is said to be systematic (also called block code) where: Input data $D = [a_1, a_2, a_3, \dots, a_k]$, The output systematic code word (n, k) is:
 $C = [a_1, a_2, a_3, \dots, a_k, c_1, c_2, c_3, \dots, c_r]$

However if the data bits are spread or changed at the output code word then, the code is said to be nonsystematic. The output of nonsystematic code word of (n, k) :
 $C = [c_2, a_1, c_1, a_3, a_2, c_3, \dots]$

- Hamming Distance (HD): It is important parameter to measure the ability of error detection. It the number of bits that differ between any two codewords C_i and C_j denoted by d_{ij} . For a binary (n, k) code with 2^k possible codewords, then minimum HD (d_{min}) is $\min(d_{ij})$,

where: $n \geq d_{ij} \geq 0$

For any code word, the possible error detection is:

$$2t = d_{min} - 1$$

For example, if $d_{min} = 4$, then it is possible to detect 3 errors or less. The possible error correcting is:

$$t = \frac{d_{min} - 1}{2}$$

So, for $d_{min} = 4$, it is possible to correct only one bit.

Example 1: Find the minimum HD between the following codewords. Also determine the possible error detection and the number of error correction bits.

$$C1 = [100110011], C2 = [111101100] \text{ and } C3 = [101100101]$$

Solution:

Here $d_{12} = 6$, $d_{13} = 4$ and $d_{23} = 3$, hence $d_{min} = 3$.

The possible error detection $2t = d_{min} - 1 = 2$.

The possible error correction $t = d_{min}-1/2 = 1$.

-Hamming Weight: It is the number of 1's in the non-zero codeword C_i , denoted by w_i . For example the codewords of $C1 = [1011100]$, $C2 = [1011001]$, $w1 = 4$, and $w2 = 3$ respectively. If we have two valid codewords- all ones and all zeros, in this case $HD = w_i$.

3. Parity check codes (Error detection):

It is a linear block code (systematic codes). In this code, an extra bit is added for each k information and hence the code rate (efficiency) is $k/(k+1)$. At the receiver, if the number of 1's is odd, then the error is detected. The minimum Hamming distance for this category is $d_{min}=2$, which means that the simple parity code is a single-bit error-detecting code; it cannot correct any error. There are two categories in this type: even parity (ensures that a code word has an even number of 1's) and odd parity (ensures that a code word has an odd number of 1's) in the code word.

Example 2: an even parity-check code of (5, 4), which means that $k=4$ and $n=5$.

Data word	Code word	Data word	Code word
0010	00101	0110	01100
1010	10100	1000	10001

4. Repetition codes:

The repetition code is one of the most basic error-correcting codes. The idea of the repetition code is to just repeat the message several times. The encoder is a simple device that repeats r times. For example, if we have a (3, 1) repetition code, then encoding the signal $m = 101001$ yields the code $c = 111000111000000111$.

Suppose we received a (3, 1) repetition code and are decoding the signal $c = 110001111$. The decoded message is $m=101$. For $(r, 1)$ repetition code, an error correcting capacity of $r/2$ (i.e., it will correct up to $r/2$ errors in any code word). In other words, the $d_{min}=r$, or increasing the correction capability depending on the r value. Although this code is very simple, it is also inefficient and wasteful because it uses only (2, 1) repetition code, which doubles the bandwidth and thus doubles the cost.

Example 3: A receiver of (5, 1) repetition code has received the following code words: 100100011010101, 100011001100111, 111000000111110:

- Determine the decoded data word and transmitted code words.
- Calculate the minimum Hamming distances and error correction capability.

Solution:

- a- It must sort each received code word into groups, each one contain 5 bits as follow:

$C_1=[10010, 00110, 10101]$, if the 1s is more than 0s, then the decoded bit is =1 and vise versa.

So that $m_1 = [001]$.

Similarly: $C_2=[10001, 10011, 00111]$ then $m_2 = [011]$

Also, $C_3=[11100, 00001, 11110]$ and $m_3 = [101]$

- b- For repetition code the minimum Hamming distance $r=5$. The error correction capability= $\left\lfloor \frac{r-1}{2} \right\rfloor = \frac{4}{2} = 2 \text{ bits}$

Example 4: A receiver of (5, 1) repetition code has been received the following code words:

(100100011010101, 100011001100111, 111000000111110)

- Determine the decoded data word and transmitted code words.
- Calculate the minimum Hamming distances and error correction capability.

Solution:

Splitting each RX code vector into a groupe of “5” bits with the application of “MAJORITY LOGIC PRINCIPLE” for each groupe as shown below:

(10010 00110 10101, 10001 10011 00111, 11100 00001 11110)										
0	0	1		0	1	1		1	0	1

- a) Decoded data word: $m1 = [001]$, $m2 = [011]$ and $m3 = [101]$. Therefore, TX (Transmitted) code- words are:

$C1 = 000000000011111$, $C2 = 000001111111111$, $C3 = 111110000011111$.

- b) Hamming distance: $d1-2=5$, $d1-3=5$, $d2-3=10$, So that $d_{min}=5$

- Error detection (E.D) = $d_{min}-1=5-1=4$ bits.

- Error correction (E.C) = $[(d_{min}-1)/2] = [(5-1)/2] = 2$.

OR Since the repetition is $r=5$, the error correction capability is $[(r-1)/2] = [4/2] = 2$ bits.

5. Convolutional Code

Convolutional coding were introduced in 1955 by Elias. Convolutional codes offer an approach to error control coding substantially different from that of block codes. Convolutional encoders are linear and time-invariant system given by the convolution of a binary data stream with generator sequences. They can be implemented by shift registers. Can achieve a larger coding gain than can be achieved using a block coding with the same complexity. The information, consisting of individual bits, is fed into a shift register for encoding.

A convolutional encoder is generally characterized in form **(n, k, K) or (k/n, K)**

Where: k is the input bits (in practice, usually $k=1$), n is the number of output bits. And K is the constraint length of the of the convolutinal code (where the encoder has $K-1$ memory elements).

S is the number of memory elements

$R_c = k/n$ is the coding rate, determining the number of data bits per coded bit.

The encoder shown is a (2, 1, 3), 1/2 code rate, and 4 possible states.

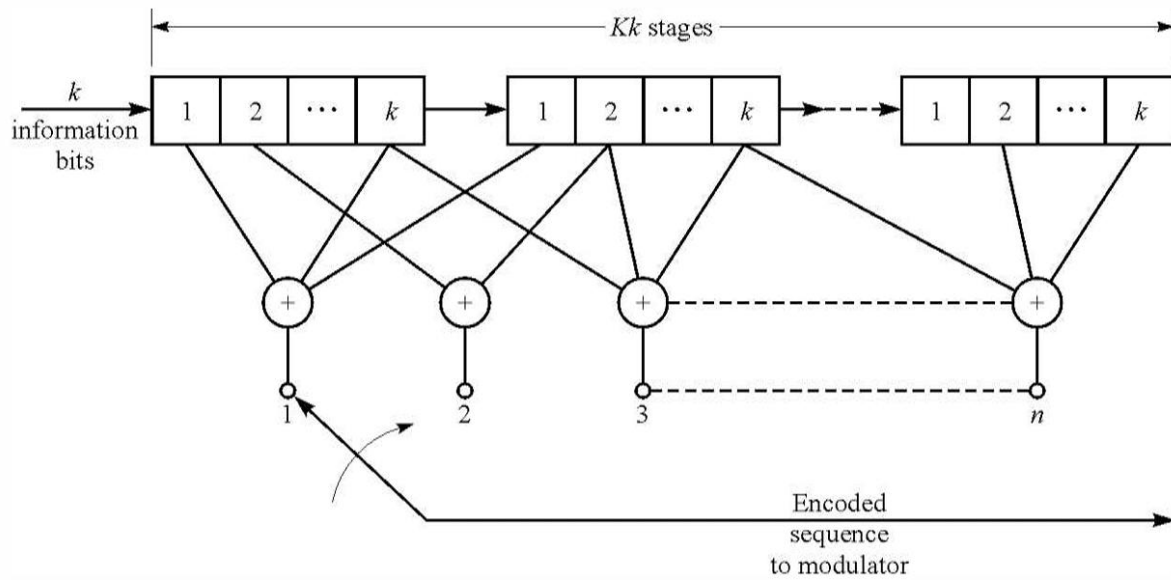


Figure 1: convolutional encoder.

The convolutional encoder (rate $\frac{1}{2}$, $K=3$) shown in Figure 2 consists of 3 shift registers, where the first one takes the incoming data bit, and the rest form the encoder's memory.

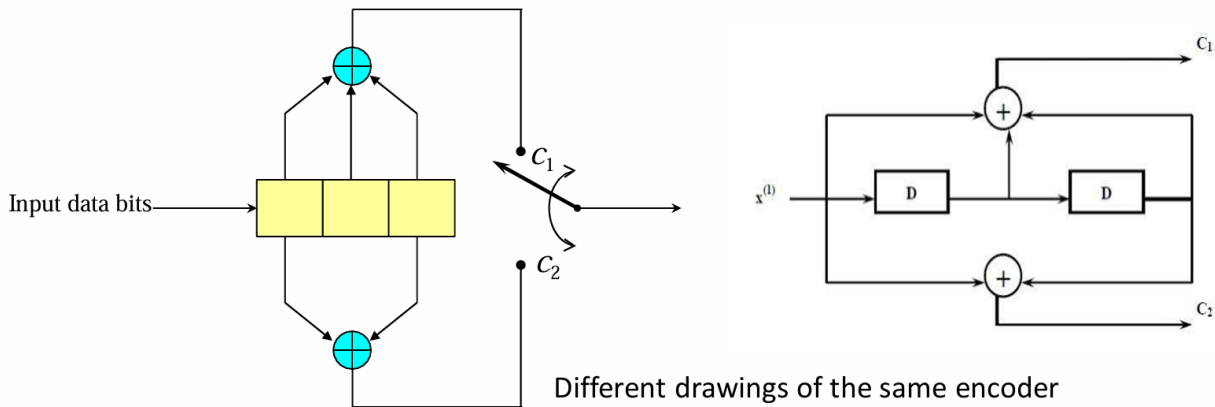


Figure 2: Convolutional encoder (rate $\frac{1}{2}$, $K=3$).

5.1.Non-Systematic Convolutional (NSC) Codes

For example, a Non-Systematic Convolutional (NSC) Code with a constraint length $K = 3$ and a generator polynomial:

$$G(D) = [1 + D, 1 + D + D^2]$$

In this case $1 + D^2$ is 101 and $1 + D + D^2$ is 111.

101 is 5_8 , where the subscript 8 indicates it is an octal number. Similarly, 111 is 7_8 . Therefore, this is the $(7, 5)_8$ NSC code.

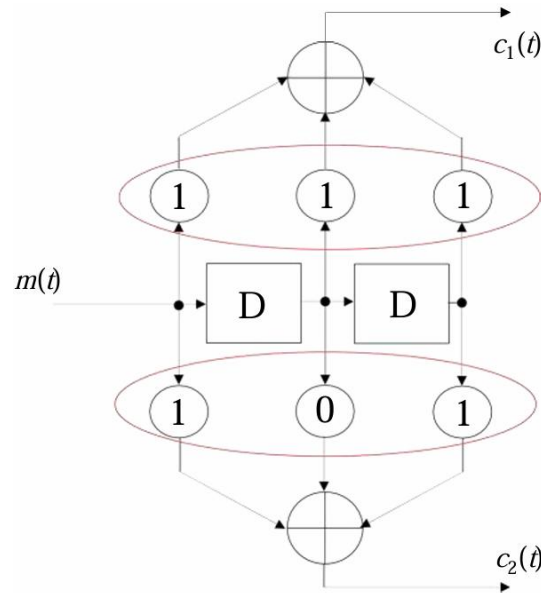


Figure 3: $(7, 5)_8$ NSC encoder.

5.2.State Table of a Convolutional $(7, 5)_8$ NSC Code

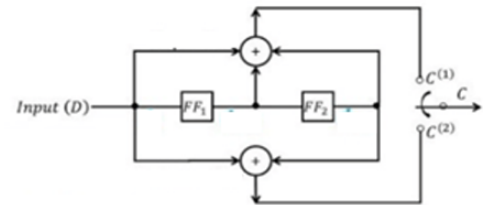
The memory elements can contain four possible values: 00, 01, 10, 11. These values are called states, and only certain state transitions are allowed. For example, we cannot go from state 00 to state 11 in one step, nor can we stay in state 10. All possible state transitions, along with their corresponding input and outputs, can be determined from the encoder and stored in a state table. The state table for the $(7, 5)_8$ NSC code is shown below:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

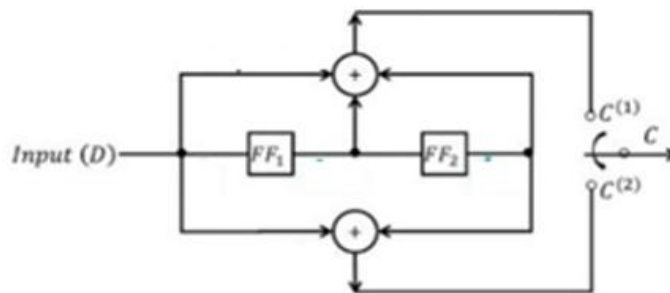
Example 5: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$ shown below. Sketch a diagram for the encoder and estimate the state table. Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0



Example 6: Consider the convolutional encoder with $k = 1$, $n = 2$ and $K = 3$ shown below. Estimate the state table to encode the message (1101100)?

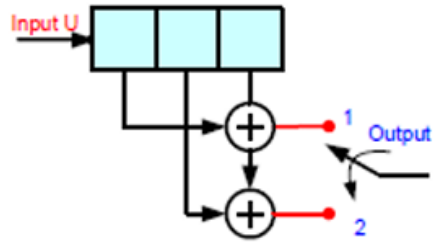


Solution:

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

Code word = 11 01 01 00 01 01 11

Example 7: An encoder diagram is shown in Figure. Find the encoder output using the state table for an input sequence 1 0 0 1 0 1 0.



Solution:

Input bit	Register Contents	State at time t_i	State at time $t_i + 1$	Output
-	000	00	00	00
1	100	00	10	11
0	010	10	01	01
0	001	01	00	11
1	100	00	10	11
0	010	10	01	01
1	101	01	10	00
0	010	10	01	01

The code word is: 11 01 11 11 01 00 01

5.3.State diagram of a Convolutional Code

A compact form of the state table is the state diagram. The state is represented by the content of the memory, i.e., the $(K-1)k$ previous bits, namely, the $(K-1)k$ bits contained in the first $(K-1)k$ stages of the shift register. Hence, there are $2^{(K-1)k}$ states.

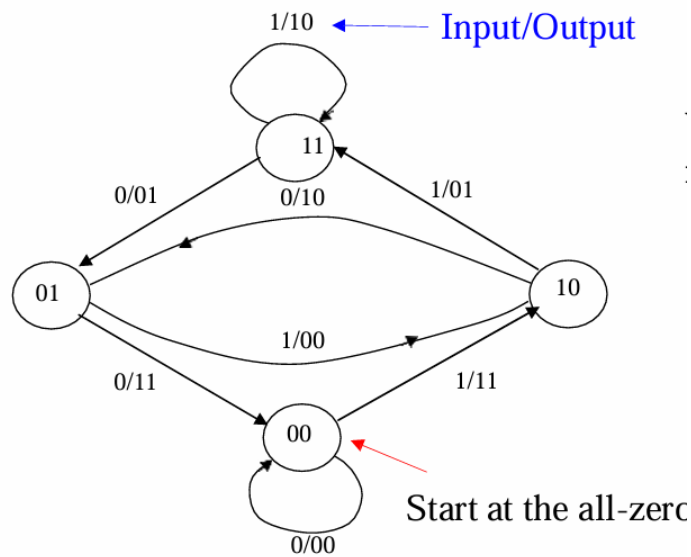
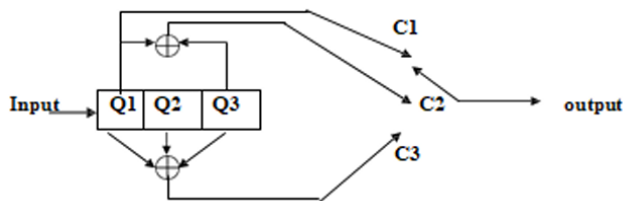


Figure 3: State diagram of a Convolutional Code.

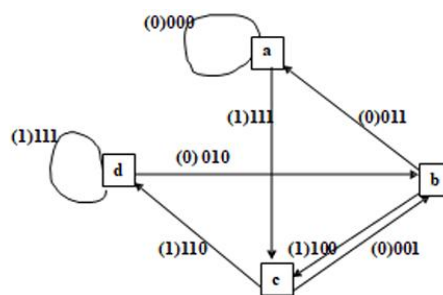
Example 8: Consider the convolutional encoder with $K=3$, $k=1$, $n=3$ shown below. Draw the state diagram of the convolutional encoder.



Solution:

Input	present state of Q2Q3	next state of Q2Q3	output C1C2C3
0	00(a)	00(a)	000
1	00(a)	10(c)	111
0	01(b)	00(a)	011
1	01(b)	10(c)	100
0	10(c)	01(b)	001
1	10(c)	11(d)	110
0	11(d)	01(b)	010
1	11(d)	11(d)	101

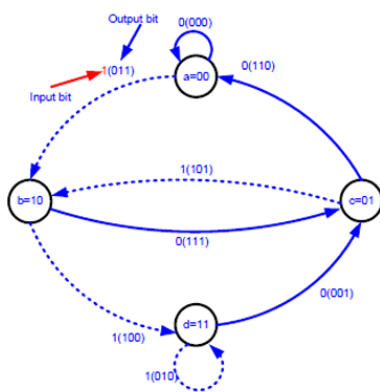
State Table



State diagram

Example 9: Estimate the state table and draw the state diagram and diagram for the $K=3$, rate= $1/3$ encoder. Consider code generated by: $g_1(X) = X+x^2$, $g_2(X) = 1+X+x^2$, $g_3(X) = 1+X$.

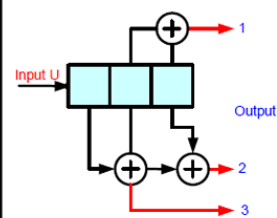
Solution:



State diagram

Input bit	State at time t_i	State at time $t_i + 1$	Output
0	00	00	000
1	00	10	011
0	01	00	110
1	01	10	101
0	10	01	111
1	10	11	100
0	11	01	001
1	11	11	010

State table



Encoder

5.4. Trellis Diagrams of a Convolutional Code

We can represent all possible state transitions with a graph, where nodes represent states and edges represent transitions. Each edge is labelled with an input and its corresponding coded output. By making copies of the graph and joining them together, we create a trellis with each path corresponding to a codeword.

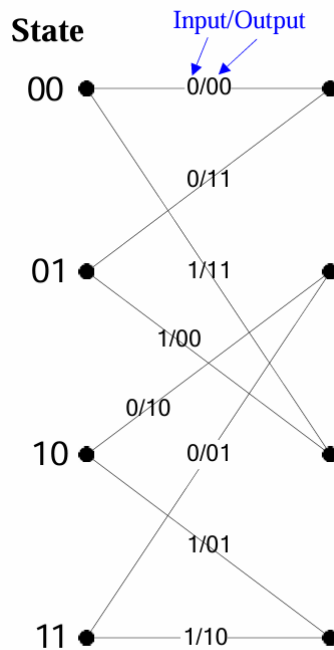
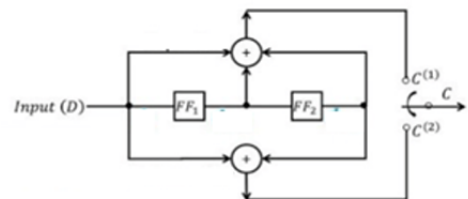


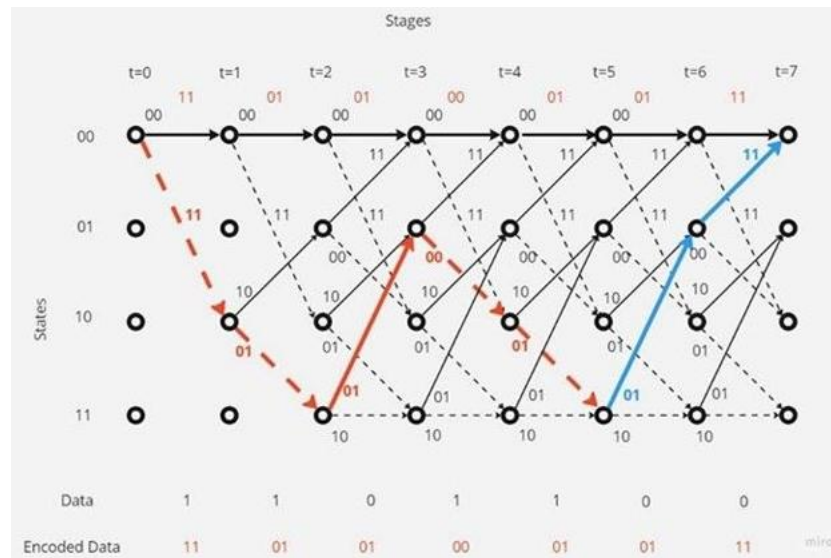
Figure 4: Trellis Diagrams of a Convolutional Code.

Example 9: Consider the convolutional encoder with $k = 1$, $n = 2$ and $K = 3$ shown below. Sketch a trellis diagram to encode the message (1101100)? Consider $g_1 = [111]$ and $g_2 = [101]$.

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0

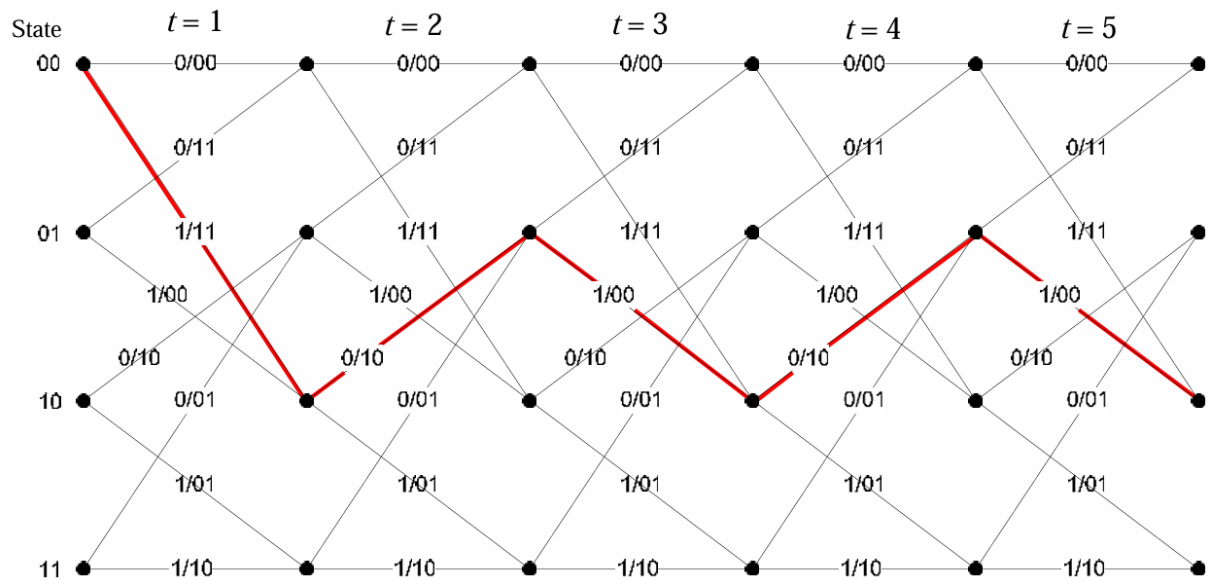


Solution:



Example 10: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$ shown below. Sketch a trellis diagram to encode the message (10101)? Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:



The codeword is: 11 10 00 10 00.

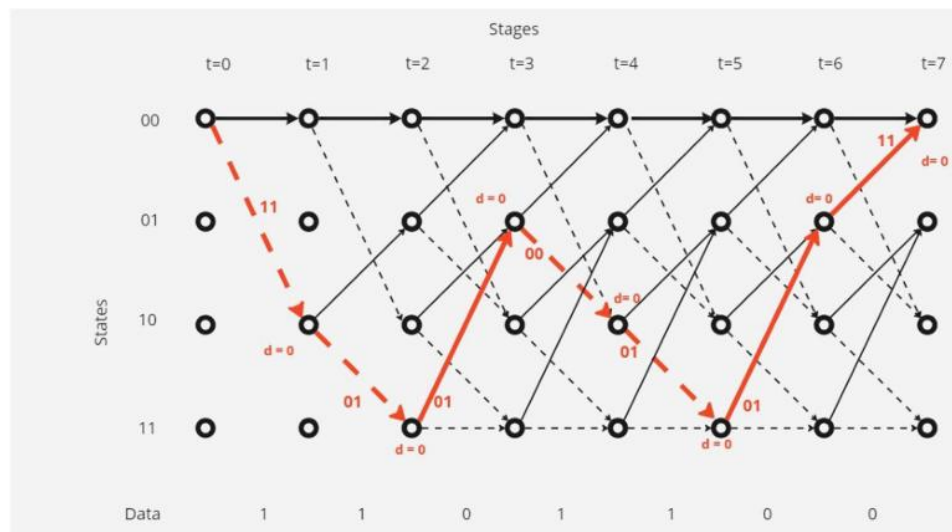
6. Viterbi's Algorithm

As stated previously, it would be impractical to compare every possible codeword with the received word until the codeword with the minimum Hamming (or Euclidean) distance is found.

One maximum-likelihood decoding algorithm, which significantly reduces the number of codewords compared with the received word, is Viterbi's algorithm, introduced by Andrew Viterbi in 1967. Viterbi's algorithm is applied to the code's trellis diagram. It is a recursive algorithm that calculates the accumulated distance between the received word and paths in the trellis at each state. When two paths converge into a state, the least likely path is discarded. Finally, the path of length n with the smallest accumulated distance is the codeword that maximizes $P(r|c)$.

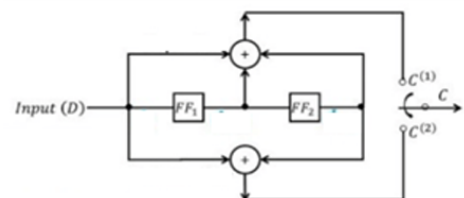
Example 11: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$. Sketch a trellis diagram to decode the codeword (11010100010111) using hard-Decision Viterbi Decoding? Consider $g_1 = [111]$ and $g_2 = [101]$.

Solution:



Example 12: Consider the convolutional encoder with $k = 1$, $n = 2$, and $K = 3$, the codeword (11010100010111). During the transmission, the 11th bit is flipped. Prove by using hard-Decision Viterbi Decoding, whether the receiver can or not correct the one-bit error? Consider $g_1 = [111]$ and $g_2 = [101]$.

Input	Initial State		Next State		Output	
	S_1	S_2	S_1	S_2	c_1	c_2
0	0	0	0	0	0	0
1	0	0	1	0	1	1
0	0	1	0	0	1	1
1	0	1	1	0	0	0
0	1	0	0	1	1	0
1	1	0	1	1	0	1
0	1	1	0	1	0	1
1	1	1	1	1	1	0



Solution:

